SatarkPay · SPHINX'26, MNIT Jaipur · PS 3.2

Project Guide

What SatarkPay is and how it works, from scratch, for everyone on the team.

Team: Ajay Yadav · Abhimanyu2211 · SaloniTrivedi11 · Harsh Verma

Everything in SatarkPay is simulated. How it would do on real payments is unknown.

Read this first

This guide explains all of SatarkPay from scratch. You do not need any machine-learning background. After reading it, you should be able to run the project, explain it to a judge and find your way around the code.

> **SatarkPay in one line:** it checks a UPI payment before the money leaves, gives it a decision (Allow, Warn, Hold or Block), and tells the person why in English and Hindi.

Read in this order

If you have	Read
10 minutes	Sections 1 to 3, then open the live link and try each test login
30 minutes	Add sections 4 to 7, and run it on your laptop
1 hour	Everything, then the part of the code you will explain, and docs/JUDGING_QA.md

1. The problem

In India, most digital payments go through UPI. Fraud on it is large: **₹981 crore lost and 12.64 lakh incidents** were reported in FY 2024-25 (MoS Finance, Lok Sabha reply, 15 Dec 2025, as reported by IANS).

The hard part is that **many scams are approved by the victim**. The right person, on the right phone, enters the right PIN. To a system that only reads numbers, such a payment can look normal. The clue is often what the person was told just before: a note, a link, or an SMS.

Our problem statement is PS 3.2, *Real-Time Fraud Detection for UPI and Digital Payments*.

The scams, in plain words

Scam	What happens
Fake collect	A request says "Refund of ₹2,999, approve to receive". It sounds like incoming money, but approving it sends money out. NPCI ordered P2P collect stopped by 1 Oct 2025, so scammers moved to merchant-style accounts.
Phishing link	A fake SMS such as "update your KYC" leads to a fake page that takes a payment.
"Scan to receive" QR	A buyer on a resale site says "scan this to receive your money". A scan and PIN only ever send money.
Mule account	A person's account is used to receive scam money and pass it on quickly, making it hard to recover.
SIM swap takeover	Someone gets control of the account and empties it from a new phone, often in another city.
"Digital arrest"	A fake police officer frightens someone into sending large amounts from their own phone, at home, in daylight. This is the hardest type.

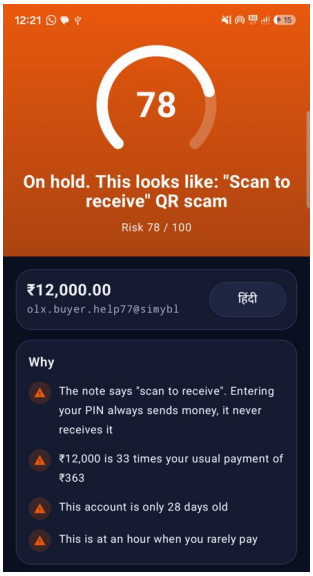
Words you will hear

Word	Meaning
UPI ID (VPA)	The address for a UPI payment, such as `name@bank`. Our simulated ones end in `@sim...`
Payer / payee	The person paying / the one receiving
Recall	Of all the scams, how many we caught
False alarm	A genuine payment we wrongly flagged
Flagged / stopped	Flagged: warned, held or blocked. Stopped: held or blocked only
XGBoost	A model that learns patterns from labelled examples
Isolation Forest	A model with no labels that notices what is unusual
SHAP	A method that shows how much each signal pushed a score up or down
SSE	A way for the server to keep sending live updates to a page
PR-AUC	One number for how well a model finds rare cases. Higher is better

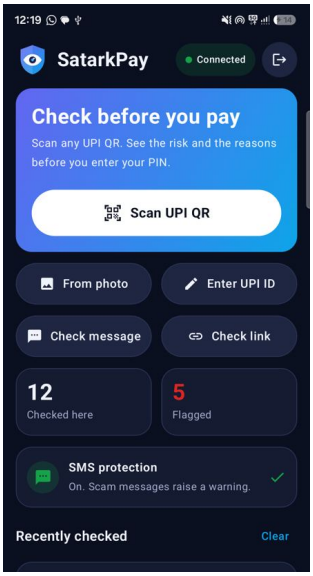
2. What we built

There are three portals and four ways to use them.

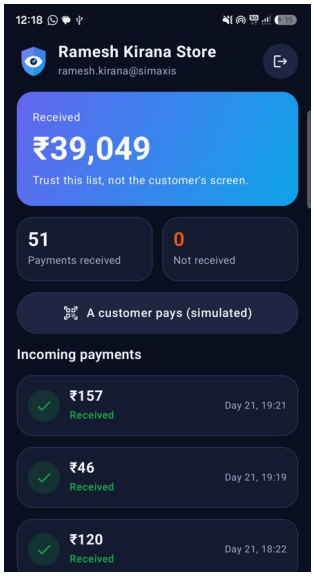
Part	Who it is for	What it does	Where
User portal	The person paying	Check a payment, see the decision and reasons, report a scam	Website `/pay`, phone app
Merchant portal	A small shop	Shows what really arrived, not a customer's screenshot	Website `/merchant`, phone app
Admin portal	A bank's fraud analyst	Live feed, alerts, reasons, money network, launch attacks	Website `/`, phone app
Scoring API	Banks and apps	One call before sending a payment, returning a decision	POST `/api/v1/score`
Phone app	Anyone	Scan QR, check messages and links, payment app handoff, SMS check	`android/`
Browser extension	Anyone	Marks risky links, warns before they open, checks Gmail	`extension/`



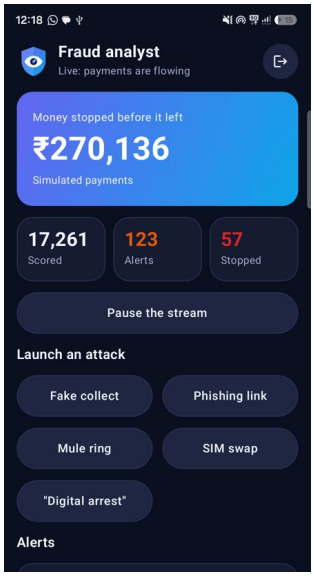
The decision and reasons



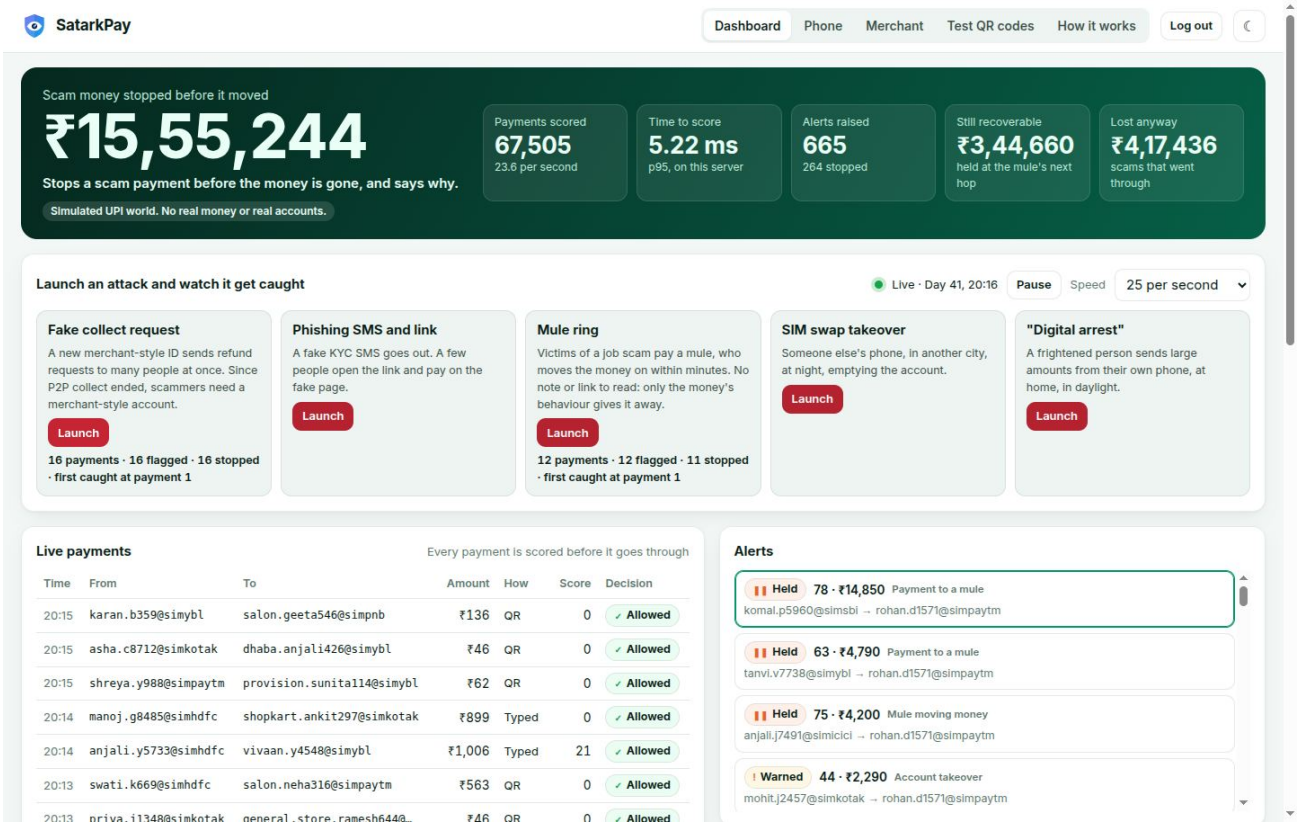
User home



Merchant



Admin



The admin dashboard on the website, during attacks

3. How one payment is checked

Every payment goes through the same steps, in milliseconds, before any money moves.

Step	What happens
1. Payment arrives	From the app, website or live stream: who pays whom, how much, the note and any link.
2. Build signals	48 features: for example, how this compares with the person's usual amount, how new the payee is, and whether money is passing straight through.

Step	What happens
3. Four layers look at it	XGBoost, Isolation Forest, the context engine and the hard rules each give an opinion.
4. Combine into one score	The formula below gives a risk, placed on a 0-100 scale.
5. Decide	Score and evidence become Allow, Warn, Hold or Block.
6. Explain	Reasons in English and Hindi, from real facts and SHAP. No language model, so nothing is invented.
7. Person acts	Pay, cancel or report. A report protects the next person, and analyst decisions become labels for later training.

The four layers

Layer	What it is good at
XGBoost	Known scam patterns in behaviour. Trained on labelled simulated payments.
Isolation Forest	Unusual payments, including types nobody has seen. Trained with no labels.
Context engine	The words: the note, the link, the UPI ID and an SMS from just before. It knows brand names and scam wording.
Hard rules	Plain facts, such as: entering a PIN never receives money.

The formula

```
risk = 1 - (1 - p) x (1 - a) x (1 - r1) x (1 - r2) ...

p  = XGBoost probability    a  = anomaly weight    r = each hard rule's weight
```

You do not need the maths. The idea: **more evidence can only raise the risk, never lower it**. Any strong layer can raise the alarm.

From score to decision

Decision	When	What the person sees
Allow	Score under 40	It goes through
Warn	Score 40 or more, or a small amount under ₹200 that would otherwise be stopped	Reasons and a 5-second pause
Hold	Score 60 or more	Reasons and a 15-second pause. Stands in for a cooling-off period and analyst review
Block	Score 80 or more, and two layers agree or a plain fact applies	Cannot continue. Report and 1930 offered

> A model's opinion alone can never refuse a payment. The rules live in `decide_action()` in `app/payments/policy.py`, and this is usually the first thing a judge asks about. The bands were set on validation data: 40 is the top 1% of genuine payments, 60 the top 0.2%, and 80 the top 0.05%.

4. The data and training

We have no access to real bank data, so **we wrote a simulator** (`ml/simulate.py`). It creates a world of people, shops and payments, and it plays 7 scam types, including hard cases such as careful scammers and genuine payments that look suspicious.

- Training: 293,416 payments. Test: 387,688 payments with 2,434 scams.
- Real fraud is rare, so the simulation has about 923 times more fraud than reality, to give the model examples.
- The live app and the training use the same feature code, so the two cannot drift apart.
- An early score of 0.99 looked too good. We treated it as a bug, found two leaks in our own data and fixed them.

5. The results

> **All of this is on simulated data that we wrote. How it does on real payments is unknown.** Always say this.

Result	Number	What it means
Scams flagged	93%	Of all simulated scams, how many got a warning or more
Scams stopped	84%	Held or blocked before the money moved
Genuine flagged	0.59%	False alarms
Fake collect, left out of training	XGBoost alone 7%, full system 81%	Our best evidence that the extra layers matter
Speed (p95)	5.06 ms	95% of requests finished faster than this, measured on a laptop
Public PaySim benchmark	0.998 to 0.697	The score drops when a shortcut in that dataset is removed. We report the lower number
QR contents, on their own	13/20 flagged, 7/20 stopped, 1/20 safe flagged	A small hand-written set. **Do not quote the main numbers for QR scans**

Why we never say "accuracy"

Reported UPI fraud is about 7 in a million payments, so a model that never flags anything is 99.999% "accurate" and useless. If fraud is that rare, even a model this good would give about **929 false alarms for every real scam**. That is why a warning must be cheap, a block must be rare, and the person always has a way out.

6. Run it

```
./run.sh            # start, then open http://localhost:8000
./run.sh wifi       # let phones on the same Wi-Fi join
./run.sh phone      # keep an Android phone connected over USB
./run.sh demo       # test pages for the browser extension
./run.sh test       # run all tests
./run.sh accounts   # make new login passwords
./run.sh train      # rebuild the data and retrain (about 2 minutes)
./run.sh evaluate   # measure everything again (about 4 minutes)
```

Or skip the setup and open **https://satarkpay.vercel.app**. Press a test login. The Vercel copy can reset its sign-ups, alerts and numbers, so the full demo runs on the laptop.

Logins

Login	Demo username	Can use
User	`sunita`	Check and make simulated payments, from a demo person's account
Merchant	`ramesh`	Its own shop's payments only
Admin	`admin`	Everything

Each portal has a test login with no password, and users and merchants can sign up. No one can create an admin account. Passwords for the demo accounts are printed once by `./run.sh` and stored only as hashes in `.env`, which is not in Git. **The server checks every request**, so hiding a button is never the only protection.

The API, in one example

```
POST /api/v1/score   (after logging in)
{ "payer": "sunita.sharma@simsbi", "payee": "refund-sbi@simybl",
  "amount": 2999, "kind": "COLLECT", "note": "Refund of Rs 2999 approve to receive" }

Answer: action BLOCK, score about 93, reasons in English and Hindi, in a few milliseconds
```

7. The code map

Folder or file	What is inside
`app/main.py`	The web server and all API routes
`app/auth.py`	Logins, sign-up, test logins, sessions
`app/payments/features.py`	Turns a payment into the 48 signals
`app/payments/scorer.py`, `models.py`	Runs the models and combines them into a score
`app/payments/context.py`, `rules.py`	Reads the words and applies the hard rules
`app/payments/policy.py`	**Decides Allow, Warn, Hold or Block**
`app/payments/explain.py`	Writes the reasons in English and Hindi
`app/payments/live.py`, `scenarios.py`, `graph.py`	Live stream, attacks, money network
`app/analysis/`, `app/data/`	The text and link engine, with brand lists
`app/static/`	Website pages, in plain JavaScript
`ml/simulate.py`, `train.py`, `evaluate.py`	Simulated world, training, measuring
`ml/qr_eval.py`	The separate QR measurement
`models/`	Trained models and measured results
`android/`	The phone app, in Kotlin. Main screens: `MainActivity.kt`, `PayScreens.kt`, `RoleScreens.kt`
`extension/`	The browser extension: links, pages, Gmail
`tests/`	All laptop tests
`docs/`	Deck, demo script, judge Q&A, reuse record, these guides

8. The other tools

Phone app

- **Scan UPI QR**: reads the code on the phone itself, shows what it says, then asks the server to check it.
- **Open payment app**: for a real UPI ID, opens the person's own UPI app. They enter their PIN there, and SatarkPay never sees it. It never moves money itself.
- **Real UPI IDs are never shown as safe**. We have no history for them, so the banner is grey and says it is not a guarantee.
- **SMS protection**: with permission, sends incoming SMS to the team's laptop, never to a public server.

Browser extension

- Marks risky links on a page, and shows a warning page before opening one.
- Shows a bar above an opened email in Gmail in a browser.
- Only talks to the server on the same computer. If that is off, it blocks nothing.

Where it runs

The laptop version is the full product: the stream runs all the time and the data stays. The Vercel copy is a public test link. Vercel functions restart and only `/tmp` can be written, so state there does not last.

9. What it does not do

Say these before a judge finds them. Honest limits build trust, and judges will test claims.

- **Everything is simulated.** There is no bank connection and no real payment data, so real-world performance is unknown.
- **QR results are weak**: it misses prize and lottery wording, a power-cut threat, and plain small payments with no words.
- **Not confirmed yet**: live camera scan, a real SMS, real Gmail, and a real UPI handoff. See the Testing Guide.
- **Demo logins**: test logins are open to anyone and reach only simulated data. There are no real accounts, and sign-ups share a simulated wallet.
- **Kafka is not built**: a simple stream simulator is used, which the problem statement allows.
- **One bank cannot see everything**: signals such as many people paying one account need a view across banks.
- **The scam label is a best guess**: the same QR can be named differently between runs. The reasons are exact, but the label is not.
- **Reused code**: parts come from an earlier project, listed in `docs/REUSED_CODE.md`. Ask the organisers whether this is allowed. If a judge asks, tell the truth.

10. Likely judge questions

They ask	You answer
Why not just use a model?	Many scams look normal in the numbers, because the victim approves them. We also read the words. When fake collect was left out of training, XGBoost alone caught 7% and the full system 81%, on simulated data.
Is it accurate?	We do not report accuracy, because it is misleading when scams are this rare. We report how many scams we catch at a fixed false-alarm level.
Does it work on real payments?	We do not know yet. Everything is simulated. A bank would start in shadow mode, which stops nothing, to find out.
Who decides when to block?	A rule, not a model alone. It takes a high score plus two agreeing layers or a plain fact. See `policy.py`.
What about false alarms?	If real fraud is this rare, most alerts would be false. So a warning is cheap, a block is rare, and the person keeps a way out.
Did you build it in 24 hours?	No. It builds on earlier code, and we can show exactly which. Say this plainly.
How is it different from what exists?	FRI rates phone numbers and MuleHunter rates accounts. We decide on one payment and explain why. Never say nobody else does this.

The full list is in `docs/JUDGING_QA.md`.

11. Your first steps

- Open **https://satarkpay.vercel.app** and try all 3 test logins.

- Clone the repo, run `./run.sh`, and launch an attack from the dashboard.
- Read `app/payments/policy.py`. It is short, and it is the heart of the system.
- Read the part of the code you will explain. Be able to explain it without notes.
- Run your share of the **SatarkPay Testing Guide**.
- Practise `docs/DEMO_SCRIPT.md` with the team.

> Never say: "99% accurate", "nobody else does this" or "it works on real UPI".